AI Incident Report Assistant(PlantGuard AI)

Problem

Safety personnel in industrial organizations cannot report incidents fast or completely, because reporting means stopping to fill a long form in the middle of the emergency.

Who has the problem? Safety personnel and supervisors on manufacturing and industrial sites who must report incidents on the shop floor.

What are they struggling with? Reporting a leak, machine fault, fall or fire while surrounded by noise, wearing gloves and responding to the situation.

User problem	Root cause	Impact
Reports are filed late, or with missing details	Typing long forms is slow and awkward during an emergency	Safety teams get incomplete data and respond slower
Voice tools fail on stuttered, noisy speech	Generic tools assume clean audio and cannot handle noise or disfluency	Workers give up and go back to paper, or skip the report
Cloud tools do not suit the site	Weak connectivity and sensitive incident data	Reports are blocked, or private data leaves the plant

Why it matters. Fast, accurate incident records are how safety teams find root causes and stop repeat accidents. A late or vague report can mean the same hazard hurts someone again.

If we do nothing. Incidents stay under-reported and incomplete, investigation stays slow, and the same hazards keep recurring, with higher injury risk and compliance exposure.

Proposed solution

We are building an AI voice assistant for industrial safety incident management: a worker talks, and the phone turns it into a complete, structured incident report, fully on the device with no cloud.

- **Tap and talk.** One tap to start describing what happened, in your own words.
- **It understands.** The assistant fills in type, severity, location, time and details by itself.
- **It asks only what is missing.** A short follow-up, never a long form.
- **Review and fix.** Check the filled form and correct anything by voice or by typing.

- **Hear it back.** The assistant reads the report aloud before you submit.

Features

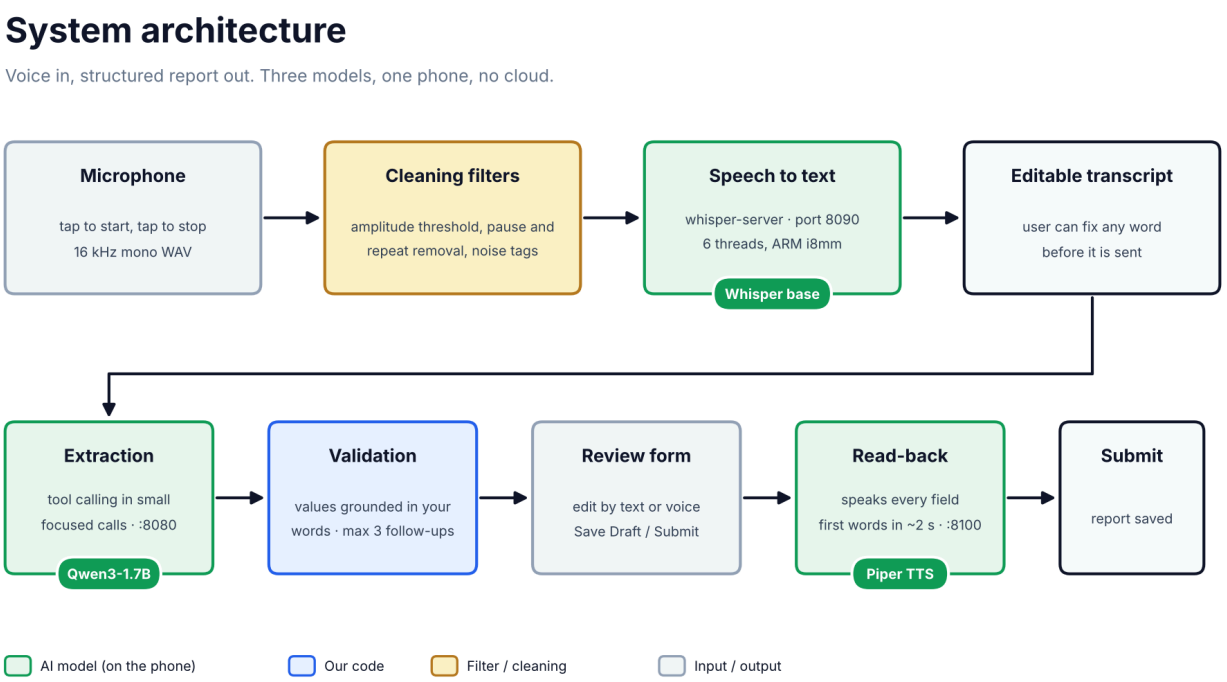
- **Voice assistant.** Speech to text and text to speech, both offline on the phone.
- **Automatic incident extraction.** Fields are filled from natural speech, with follow-up questions for anything missing.
- **Recent incidents.** Drafts stay editable by voice and text, and submitted reports are locked.

Approach and architecture

Our approach: three small models run as local servers on the phone's CPU, the models understand the speech and the text, and our code only validates and formats.

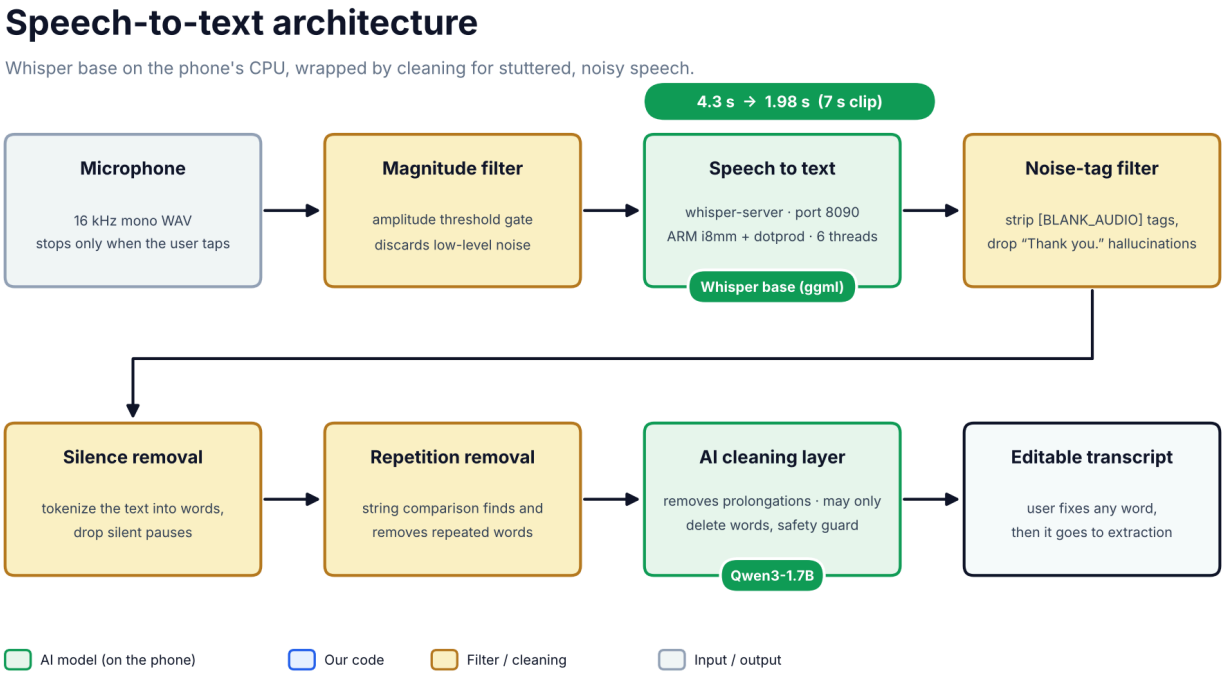
- **Models understand, code validates.** No keyword rules decide anything; every extracted value must appear in the person's own words.
- **Small focused calls.** A 1.7B model has a short context, so one long prompt is replaced by several small calls that run only when needed.
- **Tools for time.** The model picks a time function and our code does the date arithmetic.
- **Everything on the device.** Whisper, Qwen3 and Piper run offline on the Snapdragon 8 Gen 3, so nothing leaves the site.

Overall system



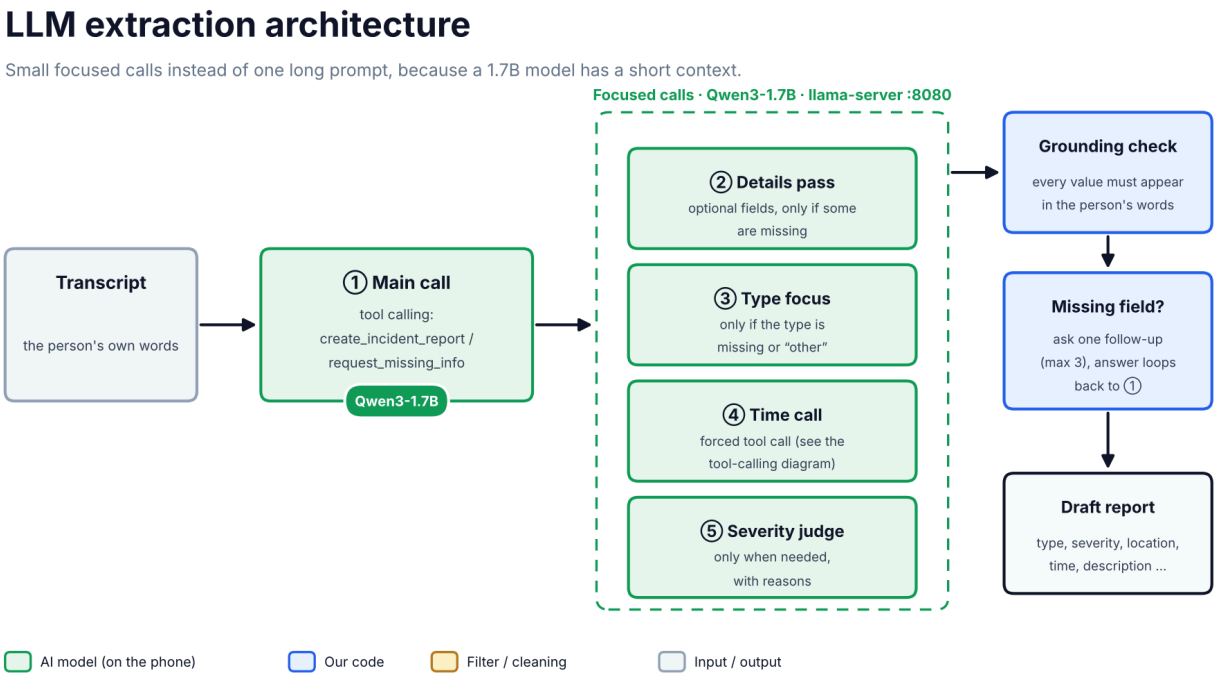
The voice goes through speech to text, extraction, validation and the review form, and can be read back before submit.

Speech to text: Whisper base



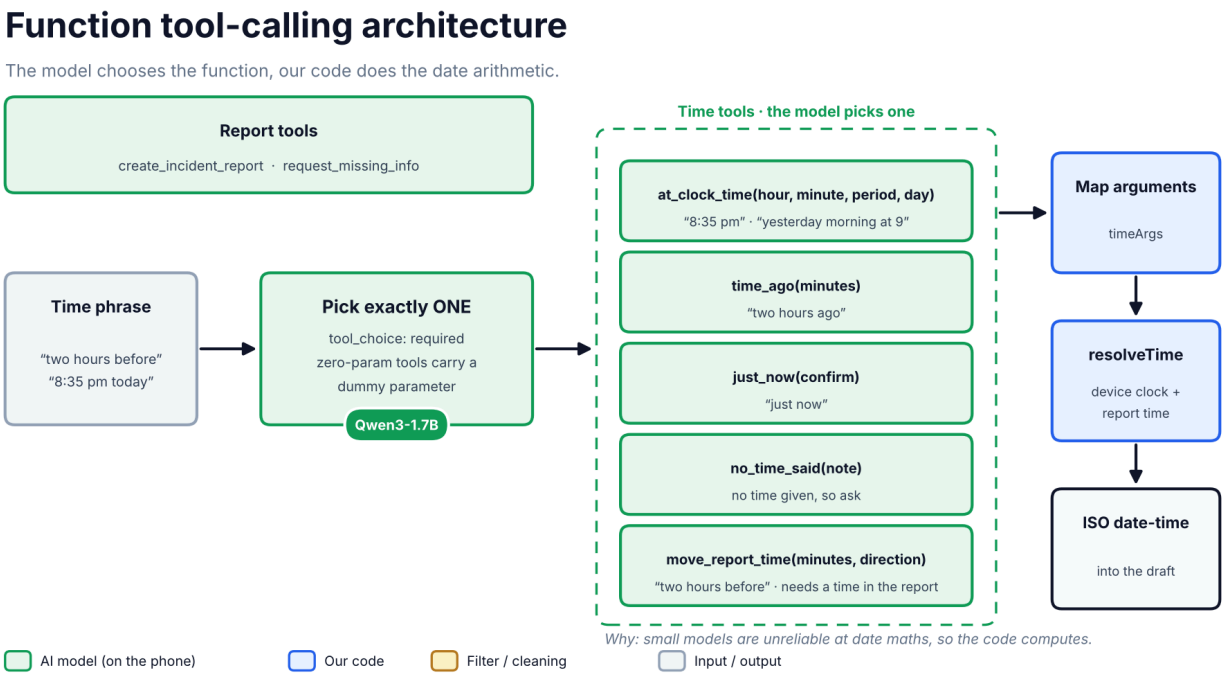
Whisper base runs in whisper-server with ARM i8mm and dotprod on 6 threads. Filters and the Qwen cleaning layer remove noise, pauses, repeats and prolongations, and the user can still edit the transcript.

LLM extraction: Qwen3-1.7B



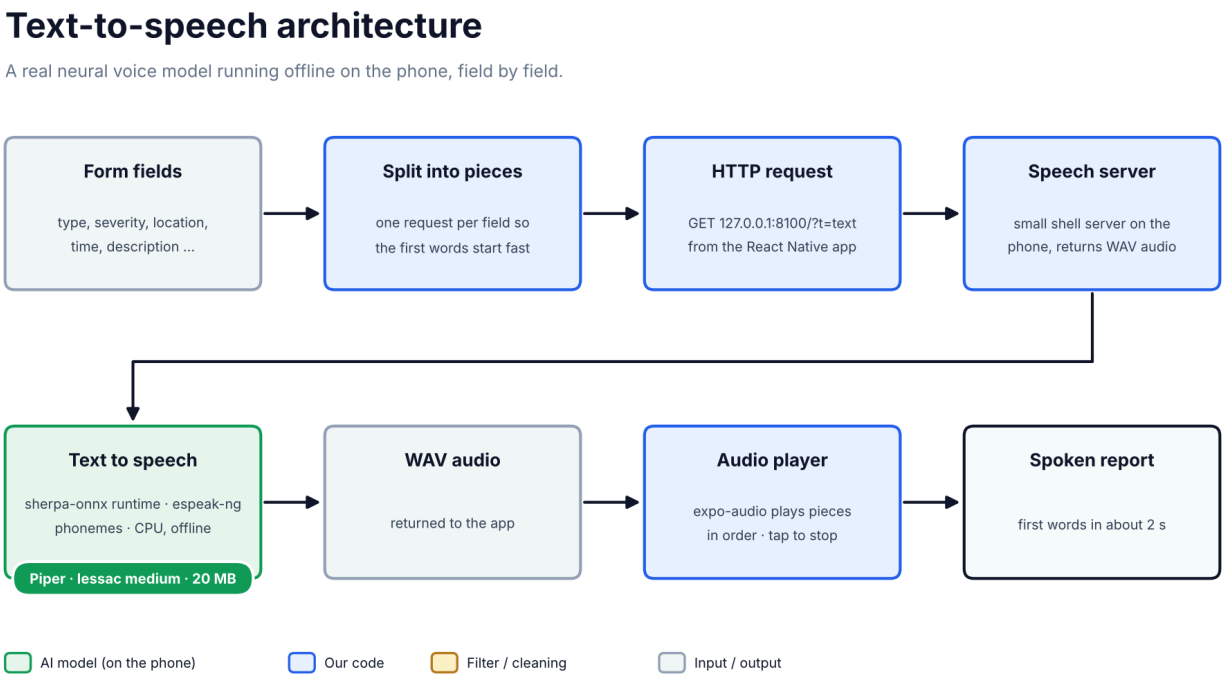
Qwen3-1.7B (Q4_K_M) runs in llama-server. A main call extracts the report, then details, type, time and severity calls run only when needed, followed by a grounding check and at most three follow-up questions.

Function tool calling



The model chooses exactly one time function, such as at_clock_time, time_ago or move_report_time. Our code maps the arguments and does the arithmetic against the device clock and the time already in the report.

Text to speech: Piper



The app sends each field to a small speech server on the phone. Piper (lessac medium, int8, about 20 MB) runs through sherpa-onnx and returns audio that the app plays piece by piece.

Results

Speech to text got 2.2x faster and extraction nearly 2x faster, and the whole assistant runs offline on the phone.

Area	Result	Why it improved
Cleaning layer	20/20 stutter cases, 8/8 safety-guard checks	Amplitude threshold filtering, pause and repeat removal by tokenization and string comparison, and a Qwen pass that may only delete words
Speech to text	4.3 s to 1.98 s for a 7 s clip	The first build ignored the chip's int8 matrix instructions (i8mm, dotprod) and used 4 threads; rebuilt with them on 6 threads
Extraction model	About 31 tokens/s on 6 CPU threads	A 1.7B quantized Qwen3 (Q4_K_M) stays loaded next to Whisper
Prompt design	25 s to 13 s on CPU	Small focused calls replaced one long prompt that overflowed the context
Text to speech	Read-back starts in about 2 s, model about 20 MB	Piper lessac-medium voice runs offline on the device